MTH8302 – Devoir 3: Régression Linéaire Multiple

Leonard Excoffier (2085276)

Hiver 2026

Table des matières

1 Expression du modèle de régression linéaire multiple **4**
1.1 Forme scalaire 4
1.2 Notation matricielle 4
1.3 Estimateur des moindres carrés ordinaires 4

2 Chargement et préparation des données **5**
2.1 Code utilisé 5
2.2 Sorties générées 6
2.3 Commentaires 8

3 Ajustement manuel du modèle de régression multiple **9**
3.1 Code utilisé 9
3.2 Formule matricielle appliquée 10
3.3 Coefficients estimés manuellement 10
3.4 Commentaires 10

4 Comparaison avec statsmodels et scikit-learn **11**
4.1 Code utilisé 11
4.2 Comparaison des coefficients 13
4.3 Conclusion 14

5 Analyse de la variance (ANOVA) pour la régression multiple **15**
5.1 Formules mathématiques 15
5.2 Code utilisé 15
5.3 Tableaux ANOVA 18
5.4 Analyse des résultats 18

6 Analyse individuelle des coefficients avec statsmodels **20**
6.1 Code utilisé 20
6.2 Tableau ANOVA séquentiel (type I) 22
6.3 Coefficients, statistiques t et p-valeurs 23
6.4 Résumé global du modèle 23
6.5 Variables significatives et non significatives 23
6.6 Analyse et interprétation 24

7 Analyse des résidus et interprétation des résultats **25**
7.1 Code utilisé 25
7.2 Résumé numérique des résidus 27
7.3 Graphique des résidus 28
7.4 Interprétation 28

Introduction

Le dépôt GitHub [1] contient l'implémentation complète du devoir. Le code Python se trouve dans `src/`, les sorties générées automatiquement dans `output/`, et le rapport LaTeX dans `report/`. Toutes les valeurs numériques présentées ci-dessous sont injectées dynamiquement à partir des fichiers texte produits par les scripts Python.

1 Expression du modèle de régression linéaire multiple

1.1 Forme scalaire

Le modèle de régression linéaire multiple s'écrit :

$$Y_i = \beta_0 + \beta_1 X_{i1} + \beta_2 X_{i2} + \cdots + \beta_p X_{ip} + \varepsilon_i, \qquad i = 1, \ldots, n.$$

où :
— Y_i est la variable réponse,
— X_{ij} est la j-ième variable explicative pour l'observation i,
— β_0 est l'intercept,
— $\beta_1, \ldots, \beta_p$ sont les coefficients de régression,
— ε_i est l'erreur aléatoire.

Dans ce devoir, la variable réponse est `wage`, et les variables explicatives proviennent de toutes les colonnes du dataset sauf `wage` et `logwage`.

1.2 Notation matricielle

Sous forme matricielle, le modèle s'écrit :

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon},$$

avec :

$$\mathbf{Y} \in \mathbb{R}^n, \qquad \mathbf{X} \in \mathbb{R}^{n \times (p+1)}, \qquad \boldsymbol{\beta} \in \mathbb{R}^{p+1}, \qquad \boldsymbol{\varepsilon} \in \mathbb{R}^n.$$

La première colonne de $\mathbf{X}$ contient uniquement des 1 afin d'inclure l'intercept β_0.

1.3 Estimateur des moindres carrés ordinaires

L'estimateur MCO des coefficients est donné par :

$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}.$$

En pratique, l'implémentation numérique la plus stable consiste à résoudre le système linéaire

$$(\mathbf{X}^\top \mathbf{X})\hat{\boldsymbol{\beta}} = \mathbf{X}^\top \mathbf{Y}$$

avec `numpy.linalg.solve`, ce qui évite de calculer explicitement l'inverse.

2 Chargement et préparation des données

2.1 Code utilisé

```python
"""Load Wage data and create the multiple-regression design matrix."""

from __future__ import annotations

import numpy as np
import pandas as pd

from config import DATA_PATH, save_text


def load_and_prepare_data() -> tuple[pd.DataFrame, pd.DataFrame, np.ndarray]:
    """Load the dataset, save exploration files, and return X and y."""
    data = pd.read_csv(DATA_PATH)

    save_text("head.txt", data.head().to_string())
    save_text("columns.txt", "\n".join(data.columns.tolist()))
    save_text("dtypes.txt", data.dtypes.to_string())
    save_text(
        "dimensions.txt",
        "\n".join(
            [
                f"Nombre d'observations : {data.shape[0]}",
                f"Nombre de colonnes brutes : {data.shape[1]}",
            ]
        ),
    )
    save_text("missing_values.txt", data.isna().sum().to_string())

    x_raw = data.drop(columns=["wage", "logwage"])
    x = pd.get_dummies(x_raw, drop_first=True, dtype=float)
    y = data["wage"].astype(float).to_numpy()

    save_text(
        "design_matrix_info.txt",
        "\n".join(
            [
                f"Variables explicatives brutes : {x_raw.shape[1]}",
                f"Variables explicatives après encodage : {x.shape[1]}",
                f"Forme de X : {x.shape}",
                f"Forme de X avec intercept : ({len(y)}, {x.shape[1] + 1})",
                "",
                "Colonnes utilisées :",
                *x.columns.tolist(),
```

```python
            ]
        ),
    )
    save_text(
        "encoding_notes.txt",
        "\n".join(
            [
                "Les variables catégorielles sont encodées avec pd.get_dummies(drop_first=True).",
                "La variable 'region' ne génère aucune colonne supplémentaire car elle ne contient qu'une seule modalité.",
            ]
        ),
    )
    return data, x, y


def build_design_matrix(x: pd.DataFrame) -> np.ndarray:
    """Return the design matrix with an intercept column."""
    return np.column_stack([np.ones(len(x), dtype=float), x.to_numpy(dtype=float)])


def coefficient_names(x: pd.DataFrame) -> list[str]:
    """Return coefficient names in manual and library order."""
    return ["Intercept"] + x.columns.tolist()
```

2.2 Sorties générées

Premières lignes du dataset :

```
   year  age            maritl      race        education              region        jobclass
    health health_ins   logwage        wage
0  2006   18  1. Never Married  1. White     1. < HS Grad  2. Middle Atlantic   1. Industrial       1.
    <=Good      2. No  4.318063   75.043154
1  2004   24  1. Never Married  1. White  4. College Grad  2. Middle Atlantic  2. Information  2. >=Very
    Good      2. No  4.255273   70.476020
2  2003   45        2. Married  1. White  3. Some College  2. Middle Atlantic   1. Industrial       1.
    <=Good     1. Yes  4.875061  130.982177
3  2003   43        2. Married  3. Asian  4. College Grad  2. Middle Atlantic  2. Information  2. >=Very
    Good     1. Yes  5.041393  154.685293
4  2005   50       4. Divorced  1. White       2. HS Grad  2. Middle Atlantic  2. Information       1.
    <=Good     1. Yes  4.318063   75.043154
```

Colonnes du dataset :

```
year
age
maritl
race
education
```

```
region
jobclass
health
health_ins
logwage
wage
```

Types des colonnes :

```
year            int64
age             int64
maritl            str
race              str
education         str
region            str
jobclass          str
health            str
health_ins        str
logwage       float64
wage          float64
```

Dimensions du dataset :

```
Nombre d'observations : 3000
Nombre de colonnes brutes : 11
```

Valeurs manquantes :

```
year          0
age           0
maritl        0
race          0
education     0
region        0
jobclass      0
health        0
health_ins    0
logwage       0
wage          0
```

Informations sur la matrice de design :

```
Variables explicatives brutes : 9
Variables explicatives après encodage : 16
Forme de X : (3000, 16)
Forme de X avec intercept : (3000, 17)

Colonnes utilisées :
year
```

```
age
maritl_2. Married
maritl_3. Widowed
maritl_4. Divorced
maritl_5. Separated
race_2. Black
race_3. Asian
race_4. Other
education_2. HS Grad
education_3. Some College
education_4. College Grad
education_5. Advanced Degree
jobclass_2. Information
health_2. >=Very Good
health_ins_2. No
```

Note sur l'encodage :

```
Les variables catégorielles sont encodées avec pd.get_dummies(drop_first=True).
La variable 'region' ne génère aucune colonne supplémentaire car elle ne
    contient qu'une seule modalité.
```

2.3 Commentaires

Le dataset contient 3000 observations et 11 colonnes brutes. Après exclusion de `wage` et `logwage`, puis encodage one-hot des variables catégorielles avec `drop_first=True`, on obtient 16 variables explicatives effectives. La variable `region` ne crée aucune colonne supplémentaire, car toutes les observations appartiennent à la même modalité.

3 Ajustement manuel du modèle de régression multiple

3.1 Code utilisé

```python
"""Manual multiple regression with the normal equations."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd

from config import save_dataframe, save_text
from data_preparation import build_design_matrix, coefficient_names


@dataclass(frozen=True)
class ManualRegressionResult:
    """Minimal output of the manual regression fit."""

    coefficients: pd.Series
    predictions: np.ndarray


def fit_manual_regression(x: pd.DataFrame, y: np.ndarray) -> ManualRegressionResult:
    """Fit the multiple-regression model manually and save coefficients."""
    design_matrix = build_design_matrix(x)
    xtx = design_matrix.T @ design_matrix
    xty = design_matrix.T @ y
    beta = np.linalg.solve(xtx, xty)

    names = coefficient_names(x)
    coefficients = pd.Series(beta, index=names, name="Coefficient")

    save_dataframe(
        "manual_coefficients.txt",
        pd.DataFrame({"Variable": names, "Coefficient": beta}),
        float_digits=6,
    )
    save_text(
        "manual_matrix_formula.txt",
        "\n".join(
            [
                "Estimateur MCO utilisé : beta_hat = (X^T X)^(-1) X^T Y",
```

```
                "Implémentation numérique : résolution de (X^T X) beta = X^T Y avec numpy.linalg.solve.",
            ]
        ),
    )

    predictions = design_matrix @ beta
    return ManualRegressionResult(coefficients=coefficients, predictions=predictions)
```

3.2 Formule matricielle appliquée

```
Estimateur MCO utilisé : beta_hat = (X^T X)^(-1) X^T Y
Implémentation numérique : résolution de (X^T X) beta = X^T Y avec numpy.linalg.solve.
```

3.3 Coefficients estimés manuellement

```
                    Variable  Coefficient
                   Intercept -2423.329067
                        year     1.241217
                         age     0.270688
           maritl_2. Married    17.176735
           maritl_3. Widowed     2.051726
          maritl_4. Divorced     3.966589
         maritl_5. Separated    11.530144
               race_2. Black    -5.096264
               race_3. Asian    -2.814092
               race_4. Other    -6.058831
        education_2. HS Grad     7.759204
   education_3. Some College    18.340498
   education_4. College Grad    31.239802
education_5. Advanced Degree    53.948514
     jobclass_2. Information     3.570669
       health_2. >=Very Good     6.515124
            health_ins_2. No   -17.512534
```

3.4 Commentaires

Les coefficients estimés ont des interprétations conditionnelles. Par exemple, le coefficient associé à `age` mesure la variation moyenne du salaire lorsque l'âge augmente d'une année, en gardant les autres variables explicatives constantes. Les coefficients d'éducation sont particulièrement élevés, ce qui suggère un effet important du niveau de scolarité sur le salaire.

4 Comparaison avec statsmodels et scikit-learn

4.1 Code utilisé

```python
"""Compare the manual model with statsmodels and scikit-learn."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd
import statsmodels.api as sm
from sklearn.linear_model import LinearRegression
from statsmodels.regression.linear_model import RegressionResultsWrapper

from config import save_dataframe, save_text
from data_preparation import coefficient_names


@dataclass(frozen=True)
class LibraryComparisonResult:
    """Store fitted library models."""

    statsmodels_model: RegressionResultsWrapper
    sklearn_model: LinearRegression


def compare_libraries(
    x: pd.DataFrame, y: np.ndarray, manual_coefficients: pd.Series
) -> LibraryComparisonResult:
    """Fit statsmodels and scikit-learn and save comparison outputs."""
    x_with_const = sm.add_constant(x, has_constant="add")
    model_sm = sm.OLS(y, x_with_const).fit()
    save_text("statsmodels_summary.txt", model_sm.summary().as_text())

    model_sklearn = LinearRegression()
    model_sklearn.fit(x, y)

    names = coefficient_names(x)
    sklearn_coefficients = np.concatenate(
        ([float(model_sklearn.intercept_)], model_sklearn.coef_)
    )
    statsmodels_coefficients = model_sm.params.to_numpy()

    save_dataframe(
        "sklearn_coefficients.txt",
```

```python
        pd.DataFrame({"Variable": names, "Coefficient": sklearn_coefficients}),
        float_digits=6,
    )

    comparison = pd.DataFrame(
        {
            "Variable": names,
            "Manuel": manual_coefficients.to_numpy(),
            "Statsmodels": statsmodels_coefficients,
            "ScikitLearn": sklearn_coefficients,
            "|Manuel-Statsmodels|": np.abs(
                manual_coefficients.to_numpy() - statsmodels_coefficients
            ),
            "|Manuel-ScikitLearn|": np.abs(
                manual_coefficients.to_numpy() - sklearn_coefficients
            ),
        }
    )
    save_dataframe("coefficient_comparison.txt", comparison, float_digits=10)

    max_diff_sm = float(
        np.max(np.abs(manual_coefficients.to_numpy() - statsmodels_coefficients))
    )
    max_diff_sklearn = float(
        np.max(np.abs(manual_coefficients.to_numpy() - sklearn_coefficients))
    )
    save_text(
        "coefficient_differences.txt",
        "\n".join(
            [
                f"Écart absolu max (manuel vs statsmodels) : {max_diff_sm:.6e}",
                f"Écart absolu max (manuel vs scikit-learn) : {max_diff_sklearn:.6e}",
                "Conclusion : les trois méthodes donnent les mêmes coefficients à la précision numérique près.",
            ]
        ),
    )

    return LibraryComparisonResult(
        statsmodels_model=model_sm,
        sklearn_model=model_sklearn,
    )
```

4.2 Comparaison des coefficients

Résumé de la comparaison :

```
Écart absolu max (manuel vs statsmodels) : 1.229441e-07
Écart absolu max (manuel vs scikit-learn) : 1.234298e-07
Conclusion : les trois méthodes donnent les mêmes coefficients à la précision
    numérique près.
```

Tableau des coefficients pour les trois méthodes :

```
                    Variable           Manuel      Statsmodels      ScikitLearn |Manuel-Statsmodels|
    |Manuel-ScikitLearn|
                   Intercept -2423.3290667632 -2423.3290666403 -2423.3290666398         1.229441e-07
       1.234298e-07
                        year     1.2412173217     1.2412173217     1.2412173217         6.130896e-11
       6.154988e-11
                         age     0.2706884001     0.2706884001     0.2706884001         7.105427e-13
       6.877277e-13
           maritl_2. Married    17.1767351785    17.1767351785    17.1767351785         1.190870e-11
       1.200817e-11
           maritl_3. Widowed     2.0517260412     2.0517260411     2.0517260411         4.963852e-11
       5.675904e-11
          maritl_4. Divorced     3.9665889056     3.9665889055     3.9665889055         1.582068e-11
       1.597922e-11
         maritl_5. Separated    11.5301438809    11.5301438809    11.5301438809         1.953993e-12
       9.610091e-13
               race_2. Black    -5.0962638262    -5.0962638262    -5.0962638262         9.769074e-12
       9.652723e-12
               race_3. Asian    -2.8140917480    -2.8140917480    -2.8140917480         1.567280e-11
       1.531442e-11
               race_4. Other    -6.0588313507    -6.0588313507    -6.0588313507         1.736300e-11
       6.057377e-12
        education_2. HS Grad     7.7592038792     7.7592038792     7.7592038792         5.714540e-12
       4.894751e-12
   education_3. Some College    18.3404982165    18.3404982165    18.3404982165         2.380318e-13
       9.059420e-13
   education_4. College Grad    31.2398024202    31.2398024202    31.2398024202         5.925926e-12
       5.119460e-12
education_5. Advanced Degree    53.9485141095    53.9485141095    53.9485141095         1.216449e-11
       1.153211e-11
     jobclass_2. Information     3.5706688172     3.5706688172     3.5706688172         4.566125e-12
       4.540812e-12
       health_2. >=Very Good     6.5151236199     6.5151236199     6.5151236199         2.777334e-12
       2.488676e-12
           health_ins_2. No   -17.5125344126   -17.5125344126   -17.5125344126         3.534950e-12
       3.641532e-12
```

Coefficients avec scikit-learn :

```
                    Variable  Coefficient
                   Intercept -2423.329067
                        year     1.241217
                         age     0.270688
           maritl_2. Married    17.176735
           maritl_3. Widowed     2.051726
          maritl_4. Divorced     3.966589
         maritl_5. Separated    11.530144
               race_2. Black    -5.096264
               race_3. Asian    -2.814092
               race_4. Other    -6.058831
        education_2. HS Grad     7.759204
   education_3. Some College    18.340498
   education_4. College Grad    31.239802
```

```
education_5. Advanced Degree    53.948514
     jobclass_2. Information     3.570669
       health_2. >=Very Good     6.515124
            health_ins_2. No   -17.512534
```

4.3 Conclusion

Les coefficients produits par l'implémentation manuelle, `statsmodels` et `scikit-learn` coïncident à la précision numérique près. Cela valide l'implémentation matricielle manuelle du modèle MCO.

5 Analyse de la variance (ANOVA) pour la régression multiple

5.1 Formules mathématiques

La décomposition de la variabilité totale est :

$$SC_{\text{totale}} = SC_{\text{reg}} + SC_{\text{res}}.$$

avec :

$$SC_{\text{totale}} = \sum_{i=1}^{n}(Y_i - \bar{Y})^2, \qquad SC_{\text{reg}} = \sum_{i=1}^{n}(\hat{Y}_i - \bar{Y})^2, \qquad SC_{\text{res}} = \sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2.$$

Les degrés de liberté sont :

$$dl_{\text{reg}} = p, \qquad dl_{\text{res}} = n - p - 1, \qquad dl_{\text{totale}} = n - 1.$$

Les moyennes carrées sont :

$$MC_{\text{reg}} = \frac{SC_{\text{reg}}}{p}, \qquad MC_{\text{res}} = \frac{SC_{\text{res}}}{n - p - 1}.$$

La statistique de test globale est :

$$F_{\text{stat}} = \frac{MC_{\text{reg}}}{MC_{\text{res}}}.$$

Sous $H_0 : \beta_1 = \cdots = \beta_p = 0$, on a :

$$F_{\text{stat}} \sim \mathcal{F}_{p,n-p-1}.$$

La p-valeur s'obtient par :

$$p\text{-valeur} = P\left(F_{p,n-p-1} \geq F_{\text{obs}}\right) = 1 - F_{\mathcal{F}_{p,n-p-1}}(F_{\text{obs}}).$$

5.2 Code utilisé

```python
"""ANOVA computations for the multiple-regression model."""

from __future__ import annotations

from dataclasses import dataclass

import numpy as np
import pandas as pd
from scipy.stats import f

```

```python
from config import save_dataframe, save_text


@dataclass(frozen=True)
class AnovaResult:
    """ANOVA summary values and table."""

    table: pd.DataFrame
    sc_total: float
    sc_reg: float
    sc_res: float
    df_reg: int
    df_res: int
    mc_reg: float
    mc_res: float
    f_stat: float
    p_value: float


def compute_anova(y: np.ndarray, y_pred: np.ndarray, p: int) -> AnovaResult:
    """Compute the ANOVA table for a regression model."""
    n = len(y)
    y_mean = float(np.mean(y))

    sc_total = float(np.sum(np.square(y - y_mean)))
    sc_reg = float(np.sum(np.square(y_pred - y_mean)))
    sc_res = float(np.sum(np.square(y - y_pred)))

    df_reg = int(p)
    df_res = int(n - p - 1)
    df_total = int(n - 1)

    mc_reg = sc_reg / df_reg
    mc_res = sc_res / df_res

    f_stat = mc_reg / mc_res
    p_value = float(f.sf(f_stat, df_reg, df_res))

    table = pd.DataFrame(
        {
            "Source": ["Régression", "Résiduel", "Total"],
            "SC": [sc_reg, sc_res, sc_total],
            "dl": [df_reg, df_res, df_total],
            "MC": [mc_reg, mc_res, np.nan],
            "F_stat": [f_stat, np.nan, np.nan],
            "p-value": [p_value, np.nan, np.nan],
        }
```

```python
    )

    return AnovaResult(
        table=table,
        sc_total=sc_total,
        sc_reg=sc_reg,
        sc_res=sc_res,
        df_reg=df_reg,
        df_res=df_res,
        mc_reg=mc_reg,
        mc_res=mc_res,
        f_stat=f_stat,
        p_value=p_value,
    )


def save_anova_outputs(manual: AnovaResult, sklearn: AnovaResult) -> None:
    """Save ANOVA tables and a short comparison summary."""
    save_dataframe("anova_manual.txt", manual.table, float_digits=6)
    save_dataframe("anova_sklearn.txt", sklearn.table, float_digits=6)

    save_text(
        "anova_comparison.txt",
        "\n".join(
            [
                f"Écart absolu sur SC_reg : {abs(manual.sc_reg - sklearn.sc_reg):.6e}",
                f"Écart absolu sur SC_res : {abs(manual.sc_res - sklearn.sc_res):.6e}",
                f"Écart absolu sur F_stat : {abs(manual.f_stat - sklearn.f_stat):.6e}",
                f"Écart absolu sur la p-valeur : {abs(manual.p_value - sklearn.p_value):.6e}",
                "Conclusion : les tableaux ANOVA manuelle et scikit-learn sont identiques à la précision numérique près.",
            ]
        ),
    )
    save_text(
        "anova_interpretation.txt",
        "\n".join(
            [
                f"Statistique F globale : {manual.f_stat:.6f}",
                f"p-valeur globale : {manual.p_value:.6e}",
                "Interprétation : la statistique F compare la variabilité expliquée moyenne à la variabilité résiduelle moyenne.",
```

```
                "Une p-valeur très petite conduit à rejeter H0 : beta_1 = ...
    = beta_p = 0.",
            ]
        ),
    )
```

5.3 Tableaux ANOVA

ANOVA avec l'implémentation manuelle :

```
    Source           SC   dl            MC    F_stat       p-value
Régression 1.773588e+06   16 110849.229405 95.886163 2.837333e-254
  Résiduel 3.448498e+06 2983   1156.050318
     Total 5.222086e+06 2999
```

ANOVA avec `LinearRegression` de scikit-learn :

```
    Source           SC   dl            MC    F_stat       p-value
Régression 1.773588e+06   16 110849.229405 95.886163 2.837333e-254
  Résiduel 3.448498e+06 2983   1156.050318
     Total 5.222086e+06 2999
```

Comparaison numérique des deux tableaux :

```
Écart absolu sur SC_reg : 1.828186e-06
Écart absolu sur SC_res : 4.656613e-10
Écart absolu sur F_stat : 9.886492e-11
Écart absolu sur la p-valeur : 1.468929e-263
Conclusion : les tableaux ANOVA manuelle et scikit-learn sont identiques à la
    précision numérique près.
```

Résumé interprétatif :

```
Statistique F globale : 95.886163
p-valeur globale : 2.837333e-254
Interprétation : la statistique F compare la variabilité expliquée moyenne à
    la variabilité résiduelle moyenne.
Une p-valeur très petite conduit à rejeter H0 : beta_1 = ... = beta_p = 0.
```

5.4 Analyse des résultats

(a) Comparaison des tableaux. Les tableaux ANOVA du modèle manuel et du modèle scikit-learn sont identiques à la précision numérique près, ce qui est cohérent avec l'égalité déjà observée entre les coefficients et les prédictions.

(b) Signification de la statistique F_{stat}. La statistique F_{stat} compare la variabilité moyenne expliquée par le modèle à la variabilité moyenne restante dans les résidus. Une

grande valeur de F_{stat} signifie que le modèle explique une part substantielle de la variabilité de `wage` relativement au bruit résiduel.

(c) Signification de la p-valeur. La p-valeur très petite indique que l'hypothèse nulle globale $H_0 : \beta_1 = \cdots = \beta_p = 0$ doit être rejetée. Autrement dit, au moins une variable explicative contribue significativement à la prédiction du salaire.

(d) Conclusion globale. L'ensemble des variables explicatives améliore significativement la régression par rapport à un modèle ne contenant qu'un intercept. Le modèle multiple est donc utile globalement, même si cela ne signifie pas que toutes les variables individuelles sont significatives.

6 Analyse individuelle des coefficients avec statsmodels

6.1 Code utilisé

```python
"""Coefficient significance analysis with statsmodels."""

from __future__ import annotations

import numpy as np
import pandas as pd
import statsmodels.api as sm
import statsmodels.formula.api as smf
from statsmodels.regression.linear_model import RegressionResultsWrapper

from config import save_dataframe, save_text
from data_preparation import coefficient_names


def _build_formula(columns: list[str]) -> str:
    """Build a patsy formula while preserving original column names."""
    quoted_terms = " + ".join(f'Q("{column}")' for column in columns)
    return f"wage ~ {quoted_terms}"


def _r_squared(y: np.ndarray, y_pred: np.ndarray) -> float:
    """Compute R-squared without external helpers."""
    sc_total = float(np.sum(np.square(y - np.mean(y))))
    sc_res = float(np.sum(np.square(y - y_pred)))
    return 1.0 - sc_res / sc_total


def analyze_coefficients(
    x: pd.DataFrame,
    y: np.ndarray,
    manual_predictions: np.ndarray,
    statsmodels_model: RegressionResultsWrapper,
    sklearn_predictions: np.ndarray,
) -> None:
    """Save coefficient-level significance analyses."""
    formula_data = x.copy()
    formula_data["wage"] = y

    formula = _build_formula(x.columns.tolist())
    formula_model = smf.ols(formula, data=formula_data).fit()
    anova_type1 = sm.stats.anova_lm(formula_model, typ=1).reset_index()
    anova_type1 = anova_type1.rename(columns={"index": "Variable"})
    anova_type1["Variable"] = (
```

```
        anova_type1["Variable"]
        .astype(str)
        .str.removeprefix('Q("')
        .str.removesuffix('")')
    )
    save_dataframe("statsmodels_anova_type1.txt", anova_type1, float_digits=6)

    names = coefficient_names(x)
    coefficient_table = pd.DataFrame(
        {
            "Variable": names,
            "Coefficient": statsmodels_model.params.to_numpy(),
            "t_stat": statsmodels_model.tvalues.to_numpy(),
            "p_value": statsmodels_model.pvalues.to_numpy(),
        }
    )
    save_dataframe(
        "statsmodels_coefficients_pvalues.txt",
        coefficient_table,
        float_digits=6,
    )

    significant = coefficient_table[
        (coefficient_table["Variable"] != "Intercept")
        & (coefficient_table["p_value"] < 0.05)
    ].sort_values("p_value")
    non_significant = coefficient_table[
        (coefficient_table["Variable"] != "Intercept")
        & (coefficient_table["p_value"] >= 0.05)
    ].sort_values("p_value")

    if significant.empty:
        save_text(
            "significant_variables.txt", "Aucune variable significative au seuil de 5%."
        )
    else:
        save_dataframe("significant_variables.txt", significant, float_digits=6)

    if non_significant.empty:
        save_text(
            "non_significant_variables.txt",
            "Toutes les variables sont significatives au seuil de 5%.",
        )
    else:
        save_dataframe(
```

```python
            "non_significant_variables.txt",
            non_significant,
            float_digits=6,
        )

    r2_manual = _r_squared(y, manual_predictions)
    r2_sklearn = _r_squared(y, sklearn_predictions)
    r2_statsmodels = float(statsmodels_model.rsquared)

    save_text(
        "r_squared_comparison.txt",
        "\n".join(
            [
                f"R^2 manuel : {r2_manual:.6f}",
                f"R^2 statsmodels : {r2_statsmodels:.6f}",
                f"R^2 scikit-learn : {r2_sklearn:.6f}",
            ]
        ),
    )
    save_text(
        "model_overall_stats.txt",
        "\n".join(
            [
                f"R^2 : {statsmodels_model.rsquared:.6f}",
                f"R^2 ajusté : {statsmodels_model.rsquared_adj:.6f}",
                f"Statistique F globale : {float(statsmodels_model.fvalue):.6f}",
                f"p-valeur du test global : {float(statsmodels_model.f_pvalue):.6e}",
                f"AIC : {statsmodels_model.aic:.6f}",
                f"BIC : {statsmodels_model.bic:.6f}",
            ]
        ),
    )
```

6.2 Tableau ANOVA séquentiel (type I)

Variable	df	sum_sq	mean_sq	F	PR(>F)
year	1	22434.353973	22434.353973	19.406036	1.093938e-05
age	1	195044.757560	195044.757560	168.716495	1.445432e-37
maritl_2. Married	1	222839.086296	222839.086296	192.758985	1.623673e-42
maritl_3. Widowed	1	44.151482	44.151482	0.038192	0.845072
maritl_4. Divorced	1	2240.733294	2240.733294	1.938266	0.163961
maritl_5. Separated	1	658.673834	658.673834	0.569762	0.450413
race_2. Black	1	20189.657571	20189.657571	17.464342	3.011578e-05
race_3. Asian	1	7856.135907	7856.135907	6.795670	0.009183
race_4. Other	1	11568.641865	11568.641865	10.007040	0.001575
education_2. HS Grad	1	333230.001696	333230.001696	288.248700	8.721784e-62
education_3. Some College	1	83924.109090	83924.109090	72.595550	2.487272e-17
education_4. College Grad	1	0.051589	0.051589	4.462553e-05	0.994670

```
education_5. Advanced Degree    1 642614.293716 642614.293716   555.870522 7.575522e-113
     jobclass_2. Information    1  18008.145138  18008.145138    15.577302  8.102738e-05
       health_2. >=Very Good    1  32911.767098  32911.767098    28.469148  1.023295e-07
            health_ins_2. No    1 180023.110372 180023.110372   155.722556  7.118176e-35
                    Residual 2983  3.448498e+06   1156.050318
```

6.3 Coefficients, statistiques t et p-valeurs

```
                    Variable  Coefficient     t_stat      p_value
                   Intercept -2423.329067  -3.930508 8.669213e-05
                        year     1.241217   4.037356 5.541747e-05
                         age     0.270688   4.349973 1.407205e-05
           maritl_2. Married    17.176735   9.985434 4.054234e-23
           maritl_3. Widowed     2.051726   0.256298     0.797738
          maritl_4. Divorced     3.966589   1.374104     0.169513
         maritl_5. Separated    11.530144   2.380343     0.017359
               race_2. Black    -5.096264  -2.375211     0.017602
               race_3. Asian    -2.814092  -1.081008     0.279781
               race_4. Other    -6.058831  -1.069244     0.285046
        education_2. HS Grad     7.759204   3.274754     0.001070
   education_3. Some College    18.340498   7.277737 4.320162e-13
   education_4. College Grad    31.239802  12.259497 9.563527e-34
education_5. Advanced Degree    53.948514  19.189701 1.751162e-77
     jobclass_2. Information     3.570669   2.696773     0.007041
       health_2. >=Very Good     6.515124   4.585438 4.716800e-06
            health_ins_2. No   -17.512534 -12.478884 7.118176e-35
```

6.4 Résumé global du modèle

```
R^2 : 0.339632
R^2 ajusté : 0.336090
Statistique F globale : 95.886163
p-valeur du test global : 2.837333e-254
AIC : 29688.876578
BIC : 29790.984827
```

Comparaison de R^2 entre les trois méthodes :

```
R^2 manuel : 0.339632
R^2 statsmodels : 0.339632
R^2 scikit-learn : 0.339632
```

6.5 Variables significatives et non significatives

Variables significatives au seuil de 5% :

```
                    Variable Coefficient     t_stat      p_value
education_5. Advanced Degree   53.948514  19.189701 1.751162e-77
            health_ins_2. No  -17.512534 -12.478884 7.118176e-35
   education_4. College Grad   31.239802  12.259497 9.563527e-34
           maritl_2. Married   17.176735   9.985434 4.054234e-23
   education_3. Some College   18.340498   7.277737 4.320162e-13
       health_2. >=Very Good    6.515124   4.585438 4.716800e-06
```

```
                        age    0.270688   4.349973 1.407205e-05
                       year    1.241217   4.037356 5.541747e-05
        education_2. HS Grad    7.759204   3.274754     0.001070
     jobclass_2. Information    3.570669   2.696773     0.007041
         maritl_5. Separated   11.530144   2.380343     0.017359
               race_2. Black   -5.096264  -2.375211     0.017602
```

Variables non significatives au seuil de 5% :

```
          Variable Coefficient    t_stat  p_value
maritl_4. Divorced    3.966589  1.374104 0.169513
     race_3. Asian   -2.814092 -1.081008 0.279781
     race_4. Other   -6.058831 -1.069244 0.285046
 maritl_3. Widowed    2.051726  0.256298 0.797738
```

6.6 Analyse et interprétation

(a) Interprétation de R^2. Le coefficient de détermination vaut environ 0.3396. Le modèle explique donc près de 34% de la variabilité du salaire. Il s'agit d'une amélioration nette par rapport à la régression simple du Devoir 2, mais une fraction importante de la variabilité demeure non expliquée.

(b) Variables ayant un impact significatif. Les plus fortes contributions positives viennent surtout des niveaux d'éducation élevés, en particulier `education_5. Advanced Degree` et `education_4. College Grad`. Le fait d'être marié, d'appartenir à la classe d'emploi `Information`, d'avoir une meilleure santé auto-déclarée et l'effet de l'âge demeurent également significatifs. À l'inverse, l'absence d'assurance santé (`health_ins_2. No`) est associée à un effet négatif important et significatif sur le salaire.

(c) Variables semblant ne pas avoir d'influence significative. Les variables `maritl_3. Widowed`, `maritl_4. Divorced`, `race_3. Asian` et `race_4. Other` ont des p-valeurs supérieures à 0.05. Une fois les autres variables contrôlées, leurs effets partiels ne sont donc pas suffisamment distincts de zéro.

Remarque méthodologique. Le tableau ANOVA de type I est séquentiel : il dépend de l'ordre d'introduction des variables dans le modèle. Pour juger l'effet partiel d'une variable en présence de toutes les autres, les p-valeurs du résumé du modèle (`model.summary()`) sont plus pertinentes que les sommes des carrés séquentielles.

7 Analyse des résidus et interprétation des résultats

7.1 Code utilisé

```python
"""Residual plots and simple-vs-multiple regression comparison."""

from __future__ import annotations

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy import stats

from config import OUTPUT_DIR, save_dataframe, save_text


def _r_squared(y: np.ndarray, y_pred: np.ndarray) -> float:
    """Compute R-squared from predictions."""
    sc_total = float(np.sum(np.square(y - np.mean(y))))
    sc_res = float(np.sum(np.square(y - y_pred)))
    return 1.0 - sc_res / sc_total


def _fit_simple_age_baseline(
    age: np.ndarray, y: np.ndarray
) -> tuple[np.ndarray, np.ndarray]:
    """Fit the assignment-2 simple regression baseline on age only."""
    x_simple = np.column_stack([np.ones(len(age), dtype=float), age])
    beta = np.linalg.solve(x_simple.T @ x_simple, x_simple.T @ y)
    predictions = x_simple @ beta
    return beta, predictions


def _plot_residuals(
    age: np.ndarray, residuals: np.ndarray, filename: str, title: str
) -> None:
    """Generate the standard residual diagnostics figure."""
    figure, axes = plt.subplots(1, 3, figsize=(18, 5))

    axes[0].hist(residuals, bins=20, edgecolor="black", alpha=0.7)
    axes[0].set_title("Histogramme des résidus")
    axes[0].set_xlabel("Résidus")
    axes[0].set_ylabel("Fréquence")

    stats.probplot(residuals, dist="norm", plot=axes[1])
    axes[1].set_title("QQ-Plot des résidus")

```

```python
    axes[2].scatter(age, residuals, alpha=0.45)
    axes[2].axhline(0, color="red", linestyle="dashed", linewidth=1)
    axes[2].set_title("Résidus en fonction de l'âge")
    axes[2].set_xlabel("Âge")
    axes[2].set_ylabel("Résidus")

    figure.suptitle(title)
    figure.tight_layout()
    figure.savefig(OUTPUT_DIR / filename, dpi=150)
    plt.close(figure)


def analyze_residuals(
    age: np.ndarray, y: np.ndarray, multiple_predictions: np.ndarray
) -> None:
    """Save residual diagnostics and compare with the simple baseline."""
    residuals = y - multiple_predictions
    _plot_residuals(
        age,
        residuals,
        "residuals_plot.png",
        "Analyse des résidus : régression linéaire multiple",
    )

    residual_summary = pd.DataFrame(
        {
            "Statistique": [
                "Moyenne",
                "Écart-type",
                "Minimum",
                "Q1",
                "Médiane",
                "Q3",
                "Maximum",
            ],
            "Valeur": [
                float(np.mean(residuals)),
                float(np.std(residuals, ddof=1)),
                float(np.min(residuals)),
                float(np.quantile(residuals, 0.25)),
                float(np.median(residuals)),
                float(np.quantile(residuals, 0.75)),
                float(np.max(residuals)),
            ],
        }
    )
    save_dataframe("residual_summary.txt", residual_summary, float_digits=6)
```

```python

    beta_simple, simple_predictions = _fit_simple_age_baseline(age, y)
    r2_simple = _r_squared(y, simple_predictions)
    r2_multiple = _r_squared(y, multiple_predictions)

    rmse_simple = float(np.sqrt(np.mean(np.square(y - simple_predictions))))
    rmse_multiple = float(np.sqrt(np.mean(np.square(y - multiple_predictions))))

    save_text(
        "simple_vs_multiple.txt",
        "\n".join(
            [
                "Régression simple sur l'âge seulement :",
                f"beta_0 = {beta_simple[0]:.6f}",
                f"beta_1 = {beta_simple[1]:.6f}",
                f"R^2 = {r2_simple:.6f}",
                f"RMSE = {rmse_simple:.6f}",
                "",
                "Régression multiple :",
                f"R^2 = {r2_multiple:.6f}",
                f"RMSE = {rmse_multiple:.6f}",
                "",
                f"Gain de R^2 : {r2_multiple - r2_simple:.6f}",
                f"Réduction du RMSE : {rmse_simple - rmse_multiple:.6f}",
            ]
        ),
    )
```

7.2 Résumé numérique des résidus

```
Statistique       Valeur
    Moyenne 8.075555e-14
 Écart-type    33.909920
    Minimum  -100.334058
         Q1   -18.698897
    Médiane    -3.260224
         Q3    13.290138
    Maximum   212.787658
```

Comparaison avec la régression simple du Devoir 2 :

```
Régression simple sur l'âge seulement :
beta_0 = 81.704735
beta_1 = 0.707276
R^2 = 0.038274
```

```
RMSE = 40.915425

Régression multiple :
R^2 = 0.339632
RMSE = 33.904268

Gain de R^2 : 0.301358
Réduction du RMSE : 7.011158
```

7.3 Graphique des résidus

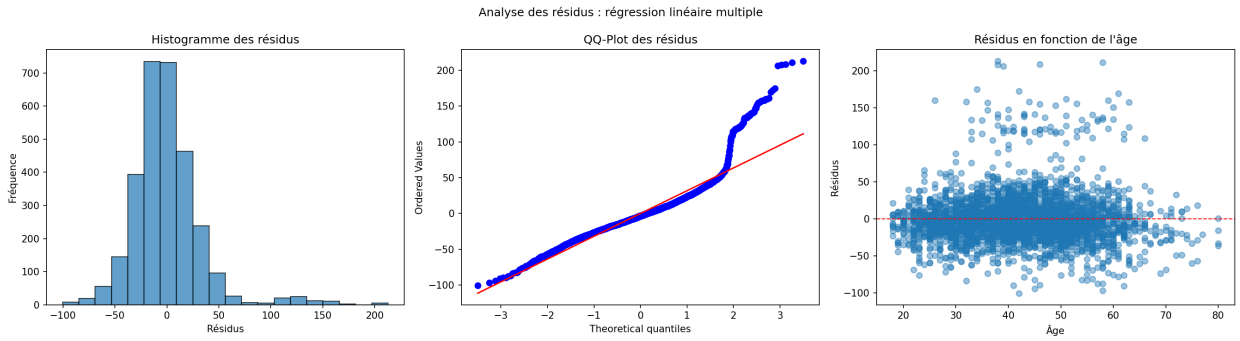

FIGURE 1 – Histogramme, QQ-plot et résidus en fonction de l'âge pour la régression multiple.

7.4 Interprétation

(a) Analyse des graphiques. L'histogramme des résidus est centré autour de zéro, mais conserve une asymétrie vers la droite. Le QQ-plot montre que les points s'écartent encore de la droite théorique dans les queues, ce qui signale la présence d'observations extrêmes et une normalité imparfaite. Le nuage des résidus en fonction de l'âge ne montre plus de tendance linéaire marquée, ce qui est meilleur qu'en régression simple, mais la dispersion n'est pas complètement uniforme.

Comparaison avec la régression simple. La régression multiple améliore sensiblement l'ajustement : R^2 passe d'environ 0.038 à 0.340 et le RMSE diminue d'environ 7 unités. Les résidus sont donc globalement plus faibles et mieux recentrés qu'avec le modèle basé uniquement sur l'âge.

(b) Conclusion sur la qualité du modèle. Le modèle multiple est clairement meilleur que le modèle simple, mais il reste imparfait. Une part importante de la variabilité du salaire n'est toujours pas expliquée, et les diagnostics de résidus suggèrent encore des violations des hypothèses de normalité et d'homoscedasticité.

(c) Pistes d'amélioration. Les pistes les plus naturelles sont :

— utiliser `logwage` comme variable réponse afin de réduire l'asymétrie ;
— ajouter des termes non linéaires, par exemple $\texttt{age}^2$;
— tester des interactions entre certaines variables (par exemple âge et éducation) ;

— considérer des méthodes de sélection de variables ou de régularisation si l'on souhaite un modèle plus parcimonieux.

Références

[1] Leonard Excoffier. *MTH8302 – Devoir 3 : Régression Linéaire Multiple*. GitHub repository. `https://github.com/excoffierleonard/mth8302-devoir3.git`. Consulté le 28 mars 2026.